



Universidad Nacional Autónoma de México

Practica 3 Compiladores 2027 - 1

Profesora: L. en C.C. Yessica Janeth Pablo Martínez

Ayudante de Laboratorio: Pedro Yamil Salazar González

Alumnos :

Marcial Ahedo Andrick Iorel

Guzmán Ramirez Diego Arturo

Equipo 06

Fecha de entrega : 21 / 09 / 2026

Objetivo

El objetivo de esta práctica fue agregar un analizador sintáctico de prácticas anteriores al analizador léxico de MiniC.
El parser recibe de manera incremental los tokens producidos por el lexer y determina si su organización corresponde con la gramática sintáctica definida para el lenguaje.
El analizador reconoce declaraciones, asignaciones, sentencias de impresión, condicionales, ciclos while, bloques y expresiones. Claro respetando la precedencia y asociatividad establecidas para los operadores de MiniC.
En esta etapa el objetivo se limita a la validación de la sintaxis. No construimos todavía un Árbol de Sintaxis Abstracta (AST) tampoco comprobaciones semánticas, como validación de tipos o verificación de declaraciones.

Evolución respecto de la Práctica 2

- Se conservaron las capacidades de reconocimiento del lexer y se modificó principalmente la forma en la que se entrega los tokens, con el objetivo de utilizarlo como entrada del nuevo analizador sintáctico.
- En la práctica anterior la interfaz pública del lexer estaba centrada en la función:
  int lexer_scan
  Para la está práctica fue necesario permitir que otro módulo solicitara los tokens uno por uno, sin que su obtención produjera salida. Por eso eliminamos lexer_scan como interfaz principal y agregamos un estado persistente para el lexer:
  ```
  typedef struct Lexer {
      FILE *source;
      size_t line;
      size_t column;
      int previous_was_cr;
  } Lexer;
  ```
- En la práctica 2, line, column y previous_was_cr existían solamente durante la ejecución de lexer_scan. En está práctica estos datos se almacenan dentro de Lexer, así conserva la posición del análisis entre diferentes solicitudes de tokens.

- lexer_init inicializa el estado del analizador, lexer_next_token continúa desde la posición previamente almacenada y devuelve únicamente el siguiente token disponible, y lexer_destroy restablece el estado utilizado por el módulo.

- Agregamos el tipo :

  ```
  typedef enum {
     LEXER_STATUS_OK,
     LEXER_STATUS_IO_ERROR,
     LEXER_STATUS_MEMORY_ERROR
  } LexerStatus;
  ```

  Esto permite diferenciar un token ERROR, que representa un error presente en el programa fuente, de una falla interna del lexer, como un problema de lectura o de memoria.

- La lógica léxica existente se conservó, aunque reorganizamos funciones auxiliares más específicas de lexer.c . Por ejemplo, agregamos read_identifier, read_integer y make_token. Las operaciones de reconocimiento de identificadores, palabras reservadas, literales booleanos, operadores simples y compuestos, enteros, comentarios de una línea y caracteres inválidos continúan utilizando las reglas desarrolladas en la práctica 2.

- Agregamos el nuevo módulo del analizador sintáctico y main.c dejó de ejecutar directamente el análisis léxico completo. En su lugar, inicializa el lexer, inicializa el parser, solicita el análisis sintáctico del programa y muestra únicamente el resultado general.

- Las 23 pruebas públicas se trasladaron a tests/lexer_public/ y las 10 pruebas propias a tests/lexer_custom/. Además, se incorporó tests/lexer_driver.c para poder solicitar tokens mediante la nueva interfaz incremental y comprobar que las funcionalidades anteriores continúan produciendo los mismos resultados.

Arquitectura e interfaz

La arquitectura de está práctica mantiene una separación explícita entre las responsabilidades del archivo principal, el lexer y el parser:
Archivo-> lexer {token} -> parser -> validación
main.c conserva la propiedad del archivo fuente y se encarga de validar los argumentos de ejecución, abrir el archivo, inicializar los analizadores, iniciar el procesamiento, mostrar el resultado general y liberar los recursos. El lexer es responsable solo de transformar los caracteres del archivo en tokens mientras que el parser trabaja únicamente con esos tokens y no vuelve a leer directamente el archivo fuente.

El lexer conserva su estado mediante la estructura Lexer, que almacena el archivo, la línea, la columna y el indicador previous_was_cr, utilizado para manejar correctamente los saltos de línea \r\n.

El parser mantiene un token actual como lookahead mediante current, además de banderas como had_error, panic_mode e internal_failure, utilizadas para registrar errores, controlar la recuperación y diferenciar fallos internos del lexer.

La comunicación entre ambos módulos se realiza mediante lexer_next_token. El lexer crea el token y lo entrega al parser, que pasa a ser responsable de liberar su memoria mediante token_destroy cuando avanza o termina el análisis.

También se distinguen los errores del programa de los fallos internos. Un token ERROR representa un error léxico del código fuente, mientras que estados como LEXER_STATUS_IO_ERROR o LEXER_STATUS_MEMORY_ERROR indican problemas internos durante la lectura o manejo de memoria.

Descenso recursivo y gramática

El parser fue implementado mediante descenso recursivo, las principales producciones de la gramática de MiniC se representan con funciones específicas. El análisis inicia en parser_parse, que procesa sentencias hasta encontrar TOKEN_EOF. La función parse_statement utiliza el token actual para decidir qué tipo de sentencia debe analizarse. Cada construcción cuenta con su propia función, como parse_declaration,

parse_assignment, parse_print_statement, parse_if_statement, parse_while_statement y parse_block.
Las declaraciones permiten una inicialización opcional, las asignaciones requieren una expresión y punto y coma, y print exige una expresión entre paréntesis.

Expresiones

La gramática de expresiones se implementó mediante una función distinta para cada nivel de precedencia con está jerarquía:
parse_expression < parse_or_expression < parse_and_expression < parse_equality_expression < parse_comparison_expression < parse_additive_expression < parse_multiplicative_expression < parse_unary_expression < parse_primary

Cada nivel reconoce sus propios operadores y delega sus operandos al nivel de mayor precedencia. Así operadores como * y / se procesan antes que + y -, y estos antes que comparaciones, igualdad y operadores lógicos.

Función	Operador
parse_or_expression	||
parse_and_expression	&&
parse_equality_expression	==, !=
parse_comparison_expression	<, <=, >, >=
parse_additive_expression	+, -
parse_multiplicative_expression	*, /
parse_unary_expression	- unario
parse_primary	enteros, booleanos, identificadores y expresiones entre paréntesis

Los operadores binarios se reconocen mediante ciclos, lo que mantiene su asociatividad izquierda. El operador - unario se implementa de forma recursiva, permitiendo expresiones como ---x y conservando su asociatividad derecha.

parse_primary reconoce enteros, booleanos, identificadores y expresiones entre paréntesis. Cuando aparecen paréntesis, se analiza una expresión completa en su interior, permitiendo modificar la precedencia normal.
En esta práctica estas funciones solo validan que la expresión tenga una estructura correcta.

Diagnósticos y recuperación

El parser distingue entre errores sintácticos, errores léxicos y fallos internos del lexer.
Los errores sintácticos se reportan mediante syntax_error, indicando línea, columna, elemento esperado y token encontrado. Los diagnósticos se envían a stderr y activan had_error para evitar que el programa sea correcto.

La función parser_consume verifica los tokens obligatorios. Cuando encuentra uno diferente al que se espera, reporta el error e inicia la recuperación mediante synchronize.

La sincronización busca puntos como:
SEMICOLON, RBRACE ,INT ,BOOL ,IF ,ELSE ,WHILE ,PRINT ,IDENTIFIER ,LBRACE ,TOKEN_EOF
Si encuentra SEMICOLON, puede consumirlo y continuar. Si encuentra un token capaz de iniciar otra sentencia, se detiene sin consumirlo. RBRACE, ELSE y TOKEN_EOF también se conservan para que puedan ser procesados por la estructura que le corresponde.

El campo panic_mode evita producir repetidamente errores derivados del mismo problema, y la implementación asegura que siempre exista avance para evitar ciclos infinitos.
Los tokens ERROR producidos por el lexer reciben un único diagnóstico léxico, se marcan como error, se liberan y se continúa con el siguiente token. Así se evita que el mismo elemento produzca además un error sintáctico.

LEXER_STATUS_IO_ERROR y LEXER_STATUS_MEMORY_ERROR se consideran fallos internos y se registran mediante internal_failure.

El análisis continúa hasta alcanzar TOKEN_EOF, por lo que el programa solo se considera correcto si se consumió toda la entrada y no se detectaron errores léxicos, sintácticos ni fallos internos.

Resultados y pruebas

Además de las pruebas públicas proporcionadas con la práctica, se diseñaron 12 pruebas propias para comprobar casos adicionales del parser, incluyendo estructuras anidadas, casos límite, precedencia de operadores y recuperación ante errores.

Pruebas Propias	Resultado	Función
c01_empty_block	Válido	Reconocimiento de un bloque vacío { }.
c02_nested_control	Válido	Anidamiento de while, if, else, bloques y expresiones.
c03_if_without_block	Válido	Uso de if con una sentencia individual sin llaves.
c04_while_without_block	Válido	Uso de while con una sentencia individual sin bloque.
c05_no_semantic_checks	Válido	Verifica que P03 sólo valide sintaxis y no tipos o declaraciones previas.
c06_multiple_recovery	Inválido	Recuperación después de varios errores en un mismo archivo.
c07_lonely_else	Inválido	Detección de un else sin un if asociado.
c08_missing_brace_before_else	Inválido	Detección de una llave de cierre faltante antes de un else.
c09_unary_chain	Válido	Reconocimiento de varios operadores - unarios consecutivos.
c10_full_precedence	Válido	Combinación de todos los niveles de precedencia de operadores.
c11_expression_eof	Inválido	Expresión incompleta al llegar a TOKEN_EOF.
c12_lexical_and_syntax	Inválido	Manejo conjunto de errores léxicos y sintácticos sin detener inmediatamente el análisis.

También se mantuvieron las pruebas de regresión del lexer de la práctica anterior para comprobar que su adaptación a una interfaz incremental no afectara el reconocimiento léxico previamente implementado.

Problemas encontrados

- Lo principal fue adaptar el lexer de la práctica 2 a una interfaz incremental. En lugar de recorrer todo el archivo e imprimir los tokens con lexer_scan(), fue necesario implementar lexer_next_token() para devolver un token por llamada y conservar el estado del análisis entre llamadas.

- También fue necesario definir correctamente la propiedad y liberación de los tokens. Como ahora los tokens pasan del lexer al parser, se estableció que el parser es responsable de liberar el lexema del token actual al avanzar o finalizar el análisis.

- Otro problema fue distinguir los errores léxicos del programa de los fallos internos del lexer. Los caracteres inválidos generan un token ERROR, mientras que los problemas de lectura o memoria se representan con estados distintos. Además, el parser consume los tokens ERROR después de reportarlos para evitar generar también un error sintáctico por el mismo carácter.

- La implementación de expresiones también requirió separar la gramática en distintos niveles para respetar la precedencia y asociatividad de los operadores. De forma similar, se implementó synchronize() para poder continuar el análisis después de errores sintácticos sin detener el programa ni caer en ciclos infinitos.

- Durante la integración se corrigieron además dos problemas concretos: una definición duplicada de parse_primary() y un error de enlazado causado porque el Makefile todavía no incluía src/parser/parser.c.

División del trabajo

Dividimos el trabajo en documetación y los errores al no poder avanzar en el código

Guzmán Ramirez Diego Arturo

Hizo principalmente la parte de adaptar el lexer de la práctica 2 a una interfaz incremental, definir el manejo de tokens y memoria, realizar los ajustes necesarios en main.c. , y el reporte.pdf.

Marcial Ahedo Andrick Iorel

Implementó el parser descendente recursivo, incluyendo el reconocimiento de sentencias, expresiones con precedencia, Makefile y parte de la documentación README.md's , CHANGELOG.md.

Uso de herramientas de Inteligencia Artificial

Durante el desarrollo de la práctica se utilizó ChatGPT, de OpenAI, como herramienta de apoyo para cumplir todos los requisitos de la práctica, analizar errores de compilación, revisar alternativas de implementación/test propios, y apoyar la organización de la documentación.
Principalmente lo usamos en la revisión de la redacción para que se entendiera de forma clara el avance desde la práctica 2 al de está práctica, queríamos que los tecnicismos se usaran de forma correcta y la explicación que dimos se conectara de forma clara.
La descripción de algunos casos de prueba y dudas específicas en la implementación de la práctica que se especifican en la sección de problemas encontrados.
Para especificar las referencias utilizadas y para organizar los README.md con el formato de markdown y no modificar la plantilla que se encuentra en el ZIP de la práctica.
También la utilizamos para revisar los lineamientos de entrega con el propósito de que todo el contenido corresponda con los requisitos solicitados.

Prompts:
"Revisa el PDF de los lineamientos y dime si nos falta alguno de ellos, te adjunto el reporte para que puedas verificarlo, en caso de que falten requisitos dime cuáles."

"Ya que revisamos que las secciones son correctas, quiero que mejores el reporte, enfócate en la redacción, hazlo de forma formal y de manera que no pierda mis ideas pero que se entienda con claridad, es decir si se puede desarrollar más o no consideré algo en la redacción agregalo, fijate en todas las secciones, tomate tu tiempo"

"Tengo un lexer en C que usa lexer_scan(FILE *file) para recorrer todo el archivo e imprimir todos los tokens, pero necesito una interfaz incremental con lexer_init(), lexer_next_token() y lexer_destroy(), donde cada llamada a lexer_next_token() devuelva solamente un token y conserve la línea, columna y posición del archivo entre llamadas que es lo que estoy

haciendo, o como lo estoy intentado porque no entiendo cómo puedo refactorizar mi implementación sin perder el reconocimiento de tokens que ya funciona, quiero que me lo expliques con ejemplos en código y si es la idea correcta o en realidad no lo necesito ”

“El lexer puede encontrar caracteres inválidos del programa, pero también puede fallar por problemas de lectura o de memoria cómo conviene que los distinga un token ERROR producido por un carácter inválido de errores internos como malloc fallido o ferror()? “

“Cómo debe manejar el parser un token ERROR para reportarlo una sola vez ? no quiero que el mismo carácter genere también un error sintáctico, y con la solución que me des dame ejemplos de cuál sería la salida, con código”

“Mi parser detecta bien un error sintáctico, pero después puede quedarse en el mismo token o producir muchos errores repetidos. Lo que quiero es implementar una función que permita continuar con la siguiente sentencia después de un error.
uso los siguientes tokens;, }, int, bool, if, else, while, print, identificadores, { y EOF, pero tengo un problema, quiero evitar ciclos y sin consumir tokens } o else, según yo pueden ser necesarios para que una estructura exterior continúe su análisis no ? explicamelo, tal vez no es necesario excluirlos.
por el momento llevo esto :

```
static void synchronize(Parser *parser)
{
   while (!parser_check(parser, TOKEN_EOF))
   {
      if (parser_check(parser, SEMICOLON))
      {
         parser_advance(parser);
         return;
      }

      if (parser_check(parser, INT) ||
         parser_check(parser, BOOL) ||
         parser_check(parser, IF) ||
         parser_check(parser, WHILE) ||
         parser_check(parser, PRINT) ||
         parser_check(parser, IDENTIFIER) ||
         parser_check(parser, LBRACE))
      {
         return;
      }

      parser_advance(parser);
   }
}”
```

“Mi proyecto compila los archivos del lexer y main.c, pero al intentar enlazar tengo errores como undefined reference to parser_init, parser_parse y parser_destroy, las funciones sí están implementadas en src/parser/parser.c, qué debo revisar para que parser.c se compile y enlace con el ejecutable minic? dime si necesitas ver el error en terminal o el tree”

“Te adjunto los README para que puedas escribirlos con el formato de markdown y se vean con claridad todos los ejemplos, sobre todo organizalos”

Conclusiones

En esta práctica se logró integrar un parser descendente recursivo al lexer desarrollado anteriormente, y manteniendo una separación clara entre el análisis léxico y el análisis sintáctico.

Con la implementación comprendimos cómo las producciones de una gramática pueden representarse mediante funciones, así como lo importante que es lookahead, la precedencia de operadores y la recuperación ante errores para continuar analizando un programa después de detectar una falla.

Por ahora el parser únicamente debe reconocer y validar la estructura, dejando la construcción del AST para la etapa siguiente.

Referencias

1. Salazar González, P. Y. Compiladores — Práctica 1. Facultad de Ciencias, Universidad Nacional Autónoma de México, semestre 2027-1.
2. Salazar González, P. Y. Definición de MiniC. Facultad de Ciencias, Universidad Nacional Autónoma de México, semestre 2027-1.
3. Salazar González, P. Y. Lineamientos de Entrega de Prácticas. Facultad de Ciencias, Universidad Nacional Autónoma de México, semestre 2027-1.
4. Guía para la elaboración del reporte de la Práctica 1. Material proporcionado en el curso de Compiladores, Facultad de Ciencias, UNAM, semestre 2027-1.